School of Computer Science & Engineering

Department of Computer Science and Applications

TY BCA
Div – 'C'

Year – 2026-27

Project Based Learning - Project Report

On

Desi On The Go

Submitted By

Pranjali Athavale 1272240779

Under Guidance of

Punam Chaudhari

School of Computer Science & Engineering

Department of Computer Science and Applications

Certificate

This is to certify that, **Pranjali Athavale** has successfully completed his/her Project Based Learning work entitled **"Desi On The Go"** in partial fulfillment of TYBCA Sem – 5, for the year 2026-27. She has worked under our guidance and direction.

Date: 29.10.2026

Place: Pune

Project Guide Name	**Dr. Jalindar Gandal**	**Dr. Anuradha Kanade**
Punam Chaudhari	**Program Head**	**Program Director**

Index

Sr. No.	Contents	PAGE NO.
1	**Introduction**	4
1.1	Existing System and Need for System	4
1.2	Scope of System	5
1.3	Operating Environment - Hardware and Software	6
1.4	Brief Description of Technology Used	6
2	**Proposed System**	8
2.1	Feasibility Study	8
2.2	Objectives of Proposed System	14
2.3	Users of System	15
3	**Analysis and Design**	16
3.1	System Requirements (Functional and Non-Functional requirements	16
3.2	Module Hierarchy Diagram	17
3.3	Sample Input and Output Screens	18
4	**Coding**	27
4.1	Code snippets	27
5	**Limitations of Proposed System**	30
6	**Proposed Enhancements**	31
7	**Conclusion**	32
8	**Bibliography**	33

1. Introduction

1.1 Existing System and Need for System

Existing food recommendation platforms and social review sites focus predominantly on mass-market popularity, generic ratings, and mainstream commercial venues. Consequently, they entirely lack cultural context, community-driven discovery mechanisms, and features specifically tailored for diaspora populations seeking a connection to home. Traditional tools fail to bridge the gap for food explorers who wish to discover authentic regional cuisines grounded in shared linguistic roots, heritage, and traditional preparation methods.

The Solution: Desi on the Go

The Desi on the Go system is engineered to bridge this critical gap by seamlessly combining media sharing, interactive social feedback, location-based mapping, and deep cultural insights into a unified, user-centric platform. To support this vision, the system establishes a robust, full-scale technical architecture featuring:

- Secure User Authentication & Persistent Sessions:
  Ensuring reliable user management, profile personalization, and safe data handling.
- Optimized Asynchronous Backend Routing:
  Powered by high-performance asynchronous frameworks to guarantee low latency, rapid data retrieval, and smooth interactions.
- Integrated Cultural Discovery Ecosystem:
  Enabling users to explore traditional dishes, verified regional vendors, and authentic community stories in real-time.

1.2 Scope of System

The scope of the Desi on the Go platform covers functional, technical, operational, and structural dimensions to establish clear system boundaries.

Product Scope

The product provides a culture-focused social ecosystem for diaspora communities and regional food enthusiasts. It bridges regional culinary exploration with heritage discovery by helping users find authentic regional dishes based on shared languages and locations.

Functional Scope

Functionally, the system manages user registration, profile setup with language preferences, and persistent JWT-based sessions. Users can create posts with location tags and multi-photo collages, interact via real-time likes and comments, and access AI-driven culinary insights tailored to regional food keywords.

Technical Scope

Technically, the project encompasses a responsive single-page frontend engineered with React, TypeScript, and Tailwind CSS. The backend utilizes asynchronous Python and FastAPI REST endpoints, Pydantic for payload validation, and SQLAlchemy ORM for PostgreSQL database operations.

Operational & Deployment Scope

Operationally, the system covers local server configurations using Uvicorn ASGI, environment variable management, database schema migrations, and client-side production builds.

1.3 Operating Environment - Hardware and Software

Hardware Requirements

- **Processor:** Intel Core i3 (8th Gen) / AMD Ryzen 3 or higher
- **RAM:** 8 GB minimum (16 GB recommended for concurrent backend/frontend servers)
- **Storage:** 256 GB SSD (5 GB minimum available space for development environment and database)

Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux (Ubuntu 20.04+)
- **Development Environment / IDE:** VS Code (Visual Studio Code)
- **Runtime Environments:** Node.js (v18.0+), Python (v3.10+)
- **Database Server:** PostgreSQL 14+ / SQLite (Development)
- **API Testing & Management:** Postman / FastAPI Interactive Swagger UI

1.4 Brief Description of Technology Used

Frontend Framework & Build System (React, TypeScript & Vite):

- **React:** Chosen for its component-based architecture and efficient Virtual DOM, which enables seamless real-time state updates when users interact with posts, likes, and comments without requiring full page reloads.
- **TypeScript:** Selected to enforce strict compile-time type safety across all UI components and API response interfaces, significantly reducing runtime errors, payload mismatches, and unexpected crashes during data rendering.
- **Vite:** Utilized as a lightning-fast build tool and development server that provides Hot Module Replacement (HMR), drastically speeding up frontend development and optimizing production asset bundling.

UI Styling (Tailwind CSS):

- Chosen for its utility-first approach that accelerates UI development while maintaining a clean, highly responsive layout across mobile and desktop devices. It ensures consistent branding and dark/light design styling without the overhead of heavy custom CSS files.

Backend Framework (FastAPI & Python):

- Chosen because of its native support for asynchronous (ASGI) request processing, which handles high-concurrency requests—such as simultaneous feed fetching and media loading—with extremely low latency. Additionally, its built-in **Pydantic** models provide automatic, high-performance data validation and structured error handling for incoming JSON payloads.

Database & ORM (PostgreSQL & SQLAlchemy):

- **PostgreSQL:** Selected as the primary relational database for its enterprise-grade reliability, support for complex queries, and robust handling of relational data models.
- **SQLAlchemy:** Utilized as the Object-Relational Mapper (ORM) to cleanly map Python classes to database tables. It abstracts raw SQL operations, protects against SQL injection attacks, and manages complex relational dependencies (such as foreign keys across Users, Posts, Likes, and Comments).

Security & Authentication (OAuth2, JWT & Passlib/Bcrypt):

- **OAuth2 with JWT (JSON Web Tokens):** Implemented to provide stateless, scalable session management. Users receive an encrypted token upon login, eliminating the need to store server-side session state while securely authorizing every backend API request.
- **Passlib (Bcrypt):** Used to salt and hash user passwords before storing them in the database, ensuring industry-standard security and protecting sensitive credential data against compromise.

2. Proposed System

2.1 Feasibility Study

Before initiating the architectural design and software development phases of **Desi on the Go**, a multi-dimensional feasibility study was conducted. Evaluating a software system's viability before writing code is critical to identifying potential bottlenecks, optimizing resource allocation, and preventing operational failure. This study rigorously analyzes the platform across three core engineering pillars: **Technical Feasibility**, **Economic Feasibility**, and **Operational Feasibility**.

Technical Feasibility Analysis

The technical feasibility evaluation assesses whether the proposed technology stack, software architecture, infrastructure requirements, and integrations can reliably support the platform's performance, concurrency, security, and scalability demands without introducing critical points of failure.

Frontend Architecture & UI Rendering Engine

The client-side interface is built using React 18 paired with TypeScript and Tailwind CSS, orchestrated by Vite as the module bundler and build system.

- Component-Based Virtual DOM:
  React was selected due to its asynchronous Virtual DOM reconciliation algorithm. In a content-heavy application where users frequently scroll through photo collages, trigger like/unlike counters, and post threaded comments, React minimizes direct DOM manipulations. This maintains a steady frame rate ($60fps$) and low interaction-to-next-paint (INP) latency.
- Type-Safety & Compile-Time Safeguards:
  TypeScript introduces strict static typing across all UI components, state management stores, and API payload definitions. By catching type mismatches, missing object properties, and null-pointer exceptions at compile time, TypeScript significantly reduces runtime crashes during complex data rendering.
- Build Efficiency & HMR:
  Vite utilizes native ES modules (ESM) during development, enabling

near-instantaneous Hot Module Replacement (HMR) and optimized tree-shaking for production builds, keeping frontend bundle sizes small and fast to load.

Backend Performance & Asynchronous Execution

The application backend is engineered using FastAPI, a modern Python web framework running on top of Uvicorn, an Asynchronous Server Gateway Interface (ASGI) server.

- Asynchronous Concurrency (ASGI):
  Traditional WSGI frameworks process incoming HTTP requests synchronously, blocking worker threads during I/O operations (such as database queries or external API calls). FastAPI's native async/await execution model leverages Python's asyncio event loop. When a request awaits a PostgreSQL response or a Google Maps API payload, the thread immediately handles secondary incoming requests. This guarantees high throughput and low latency under concurrent traffic.
- Data Validation & Serialization:
  FastAPI relies on Pydantic for structural data validation. Incoming JSON payloads are automatically validated against predefined schemas before reaching business logic layers. If a client submits malformed data, Pydantic intercepts the payload and returns clear HTTP 422 error structures, eliminating edge-case bugs caused by invalid input data.

Database Architecture & Data Integrity

The persistence layer utilizes **PostgreSQL**, an enterprise-grade relational database management system, managed via the **SQLAlchemy** Object-Relational Mapper (ORM).

- **Relational Schema & Foreign Keys:** Food discovery applications rely heavily on interconnected entity relationships. PostgreSQL efficiently enforces foreign key constraints across Users, Posts, Likes, Comments, and CulturalTags tables, preventing orphaned records and maintaining ACID compliance.

- **ORM Security & Injection Protection:** SQLAlchemy abstracts raw SQL construction into parameterized Python object calls, completely neutralizing SQL injection vulnerabilities while enabling seamless schema migrations.

External Services & Hardware Requirements

The system seamlessly integrates the **Google AI (Gemini) API** for real-time cultural insights and the **Google Maps API** for spatial mapping and location services. Furthermore, the entire architecture runs efficiently on standard x86 or ARM computing hardware during development, eliminating the necessity for costly, specialized hardware accelerators or proprietary appliances.

Economic Feasibility Analysis

Economic feasibility examines the project's financial viability, initial development overheads, ongoing operational expenditures, software licensing fees, and long-term financial sustainability.

Open-Source Software Licensing & Cost Elimination

A central strategy of the Desi on the Go development pipeline is the complete utilization of open-source frameworks, software libraries, and developer tools. By avoiding proprietary enterprise software suites, licensing costs were reduced to zero across all core infrastructure layers.

Capital Expenditure (CapEx) & Operational Expenditure (OpEx)

Because development was conducted using standard workstation environments and open-source stacks, initial Capital Expenditure (CapEx) was negligible. Operational Expenditure (OpEx) is structured into clear tiers to ensure predictable financial scaling as the platform expands:

- Development Phase Costs: During initial prototyping and local staging, the application operated locally using an ngrok secure tunnel for public endpoint exposure and integration testing. This zero-cost development workflow allowed comprehensive end-to-end testing without incurring premature cloud server charges.

- Production Deployment Costs: Upon cloud deployment, operational costs are strictly utility-based and tied directly to user growth:
    - Compute Hosting: Containerized microservices hosted on cloud app platforms (such as Render, Fly.io, or AWS ECS) scale dynamically based on web traffic.
    - Database Hosting: Managed PostgreSQL instances (e.g., Supabase or AWS RDS) scale according to storage capacity and read/write IOPS.
    - API Consumption: External APIs (Google Maps and Gemini AI) operate on generous free tiers during initial rollout, transitioning to flexible pay-as-you-go pricing models as daily active user (DAU) volume grows.

Operational Feasibility Analysis

Operational feasibility assesses how effectively the platform satisfies user expectations, simplifies internal workflows, fits existing human behaviors, and achieves sustained organic adoption within target diaspora communities.

Human-Centered UI/UX Design & Adoption

Software adoption rates depend heavily on interface clarity and cognitive load. **Desi on the Go** was engineered following intuitive UI design principles to ensure that users of varying technical proficiency can navigate the platform effortlessly without formal training:

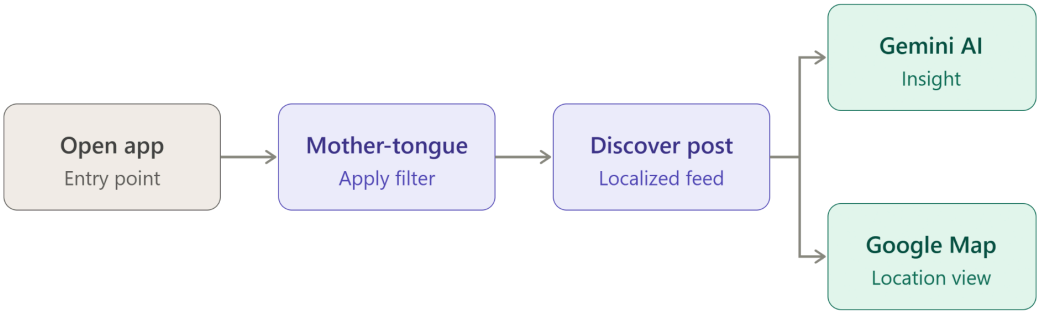
- **Familiar Social Media Paradigms:** Core interactive elements—such as media discovery feeds, double-tap liking, threaded comment sections, and user profile cards—mirror widely adopted social media standards. This visual familiarity reduces cognitive friction and enables immediate user comfort.
- **Minimalist Information Architecture:** Content layouts prioritize high-resolution imagery, clear dish titles, and distinct mother-tongue tags over cluttered navigation menus. Primary user journeys (e.g., finding a nearby dish or submitting a food post) require minimal clicks.

Diaspora Integration & Cultural Workflow Fit

Existing food review platforms cater to mainstream, mass-market dining habits, frequently misclassifying regional street foods or omitting nuanced cultural context.

Desi on the Go directly aligns with diaspora community behaviors by establishing specialized operational workflows:

- **Mother-Tongue Filtering:** Users can filter their main feed based on native languages and regional heritage (e.g., Bengali, Telugu, Punjabi, Tamil), ensuring that recommended dishes reflect exact home preferences.
- **Embedded Gemini AI Terminology Insights:** When users encounter unfamiliar regional terms (such as *Panta Bhat*, *Gongura*, or *Appam*), integrated AI triggers provide instant cultural and culinary explanations, bridging the knowledge gap between different regional communities.



Human-Centered UI/UX Design & Adoption

Software adoption rates depend heavily on interface clarity and cognitive load. Desi on the Go was engineered following intuitive UI design principles to ensure that users of varying technical proficiency can navigate the platform effortlessly without formal training:

- Familiar Social Media Paradigms: Core interactive elements—such as media discovery feeds, double-tap liking, threaded comment sections, and user profile cards—mirror widely adopted social media standards. This visual familiarity reduces cognitive friction and enables immediate user comfort.
- Minimalist Information Architecture: Content layouts prioritize high-resolution imagery, clear dish titles, and distinct mother-tongue tags over cluttered navigation menus. Primary user journeys (e.g., finding a nearby dish or submitting a food post) require minimal clicks.

System Reliability, Maintenance, & Operational Governance

From an administrative and operational standpoint, the system is designed for straightforward maintenance and moderation:

- **Stateless Authentication (JWT):** The application handles session management using stateless JSON Web Tokens. Because user state does not need to be written to server memory on every request, server restarts or scale-outs occur without disrupting active user sessions.
- **Data Moderation & Account Accountability:** Requiring JWT-authenticated accounts for posting, liking, and commenting prevents unverified spam, protects community integrity, and simplifies administrative moderation.

Feasibility Study Conclusion

The comprehensive feasibility study confirms that **Desi on the Go** is technically sound, economically sustainable, and operationally aligned with user needs. The combination of an asynchronous Python backend, a type-safe React frontend, open-source software tools, and a cultural discovery workflow guarantees a scalable, low-risk foundation for deployment and growth.

2.2 Objectives of Proposed System

The primary objective of the Desi on the Go platform is to deliver a seamless, culture-centric ecosystem that bridges regional food discovery with active diaspora community engagement. Moving beyond generic food review applications, the platform aims to culturally contextualize culinary exploration through several core objectives:

- **Culture-Centric Culinary Discovery:** Empower users to filter, search, and discover regional food posts specifically aligned with their unique linguistic roots, heritage, and mother tongue, fostering a deep sense of connection to home.
- **Real-Time Community Interaction:** Deliver robust social engagement features including dynamic liking, unliking, and threaded commenting backed by persistent session management to ensure a fluid user experience across page refreshes and browser sessions.
- **Integrated Navigation & Smart AI Insights:** Provide direct Google Maps integration for accurate, one-click restaurant and food stall locations, combined with embedded AI insight triggers powered by modern language models to explain regional food terminology, preparation styles, and historical context.
- **Architectural Precision & Data Integrity:** Eliminate API redirect overheads, payload discrepancies, and validation errors through strict RESTful routing protocols, rigorous Pydantic data validation schemas, and highly optimized database queries.

Ultimately, these objectives collectively ensure that Desi on the Go serves as more than just a directory of food spots; it establishes a scalable, full-stack foundation that balances technical reliability with rich cultural storytelling. By harmonizing efficient backend engineering with an intuitive, heritage-focused user experience, the system delivers a sustainable platform designed to support continuous feature expansion and foster long-term community growth.

2.3 Users of System

The Desi on the Go platform is engineered specifically for registered members, enforcing robust account authentication to cultivate community trust, maintain data integrity, and support personalized interactions. Users undergo registration and login procedures governed by industry-standard JSON Web Token (JWT) security protocols, establishing encrypted, persistent session state management across browser refreshes. Upon account creation, members customize their user profiles by specifying their mother tongue and regional heritage, creating a tailored baseline that dictates how discovery feeds and content filters present relevant regional cuisines.

Once successfully authenticated, these registered food explorers and contributors gain unrestricted access to the application's full functional ecosystem. Members can browse through an interactive discovery feed, author detailed culinary posts accompanied by multi-image photo collages, attach precise location coordinates, and tag specific regional dishes. Furthermore, users actively participate in community engagement through real-time liking, unliking, and threaded comment conversations, while also utilizing embedded AI-driven insight triggers to unpack complex regional food terminology, preparation methods, and historical context on demand.

By restricting content publishing, social feedback, and profile customization strictly to authenticated accounts, the system enforces user accountability and protects the community from unverified spam. This closed-loop membership model creates a reliable environment where diaspora populations and regional food enthusiasts can safely contribute authentic dining experiences, build community trust, and actively participate in preserving and sharing their vibrant culinary heritage.

3. Analysis and Design

3.1 System Requirements

System requirements define the essential operational capabilities and performance standards necessary for the **Desi on the Go** platform. These specifications ensure the application delivers a reliable user experience while maintaining structural integrity across both client-side and server-side components.

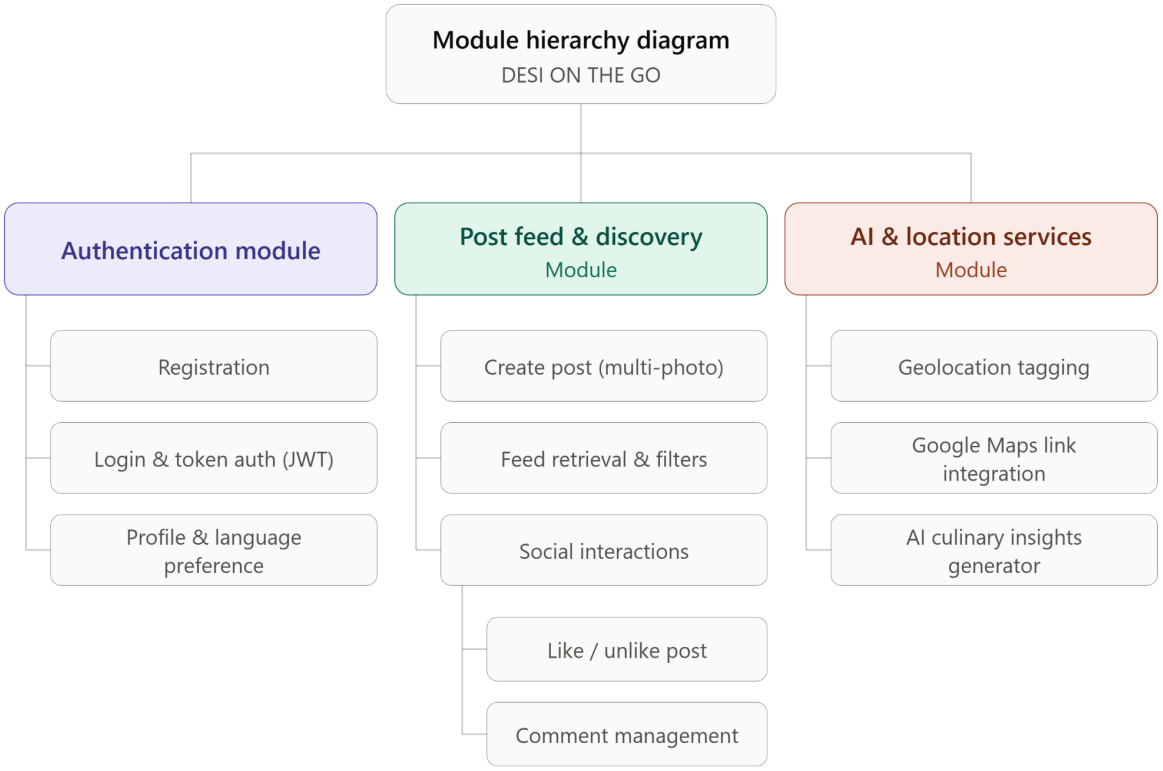
Functional Requirements

The platform requires secure JWT authentication to maintain persistent user sessions across page reloads alongside profile customization for language preferences. Registered users must be able to publish culinary posts featuring multi-photo collages and location tags, interact through real-time likes and comments, and access automated Google Maps links and AI-driven culinary explanations for regional food terminology.

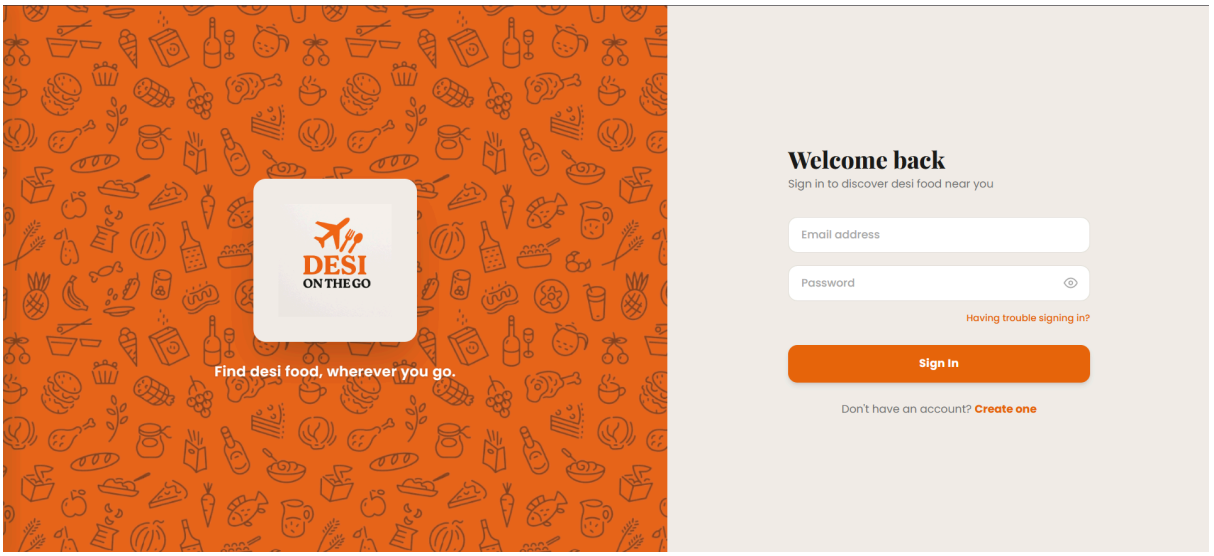
Non-Functional Requirements

The system must maintain low API latency under 200 milliseconds by utilizing FastAPI's asynchronous architecture, ensuring fast and responsive content retrieval. Security and data integrity are enforced through Bcrypt password hashing, Pydantic request payload validation, and CORS restrictions, while the React and Tailwind CSS frontend provides a clean, fully /responsive, and intuitive interface across devices.

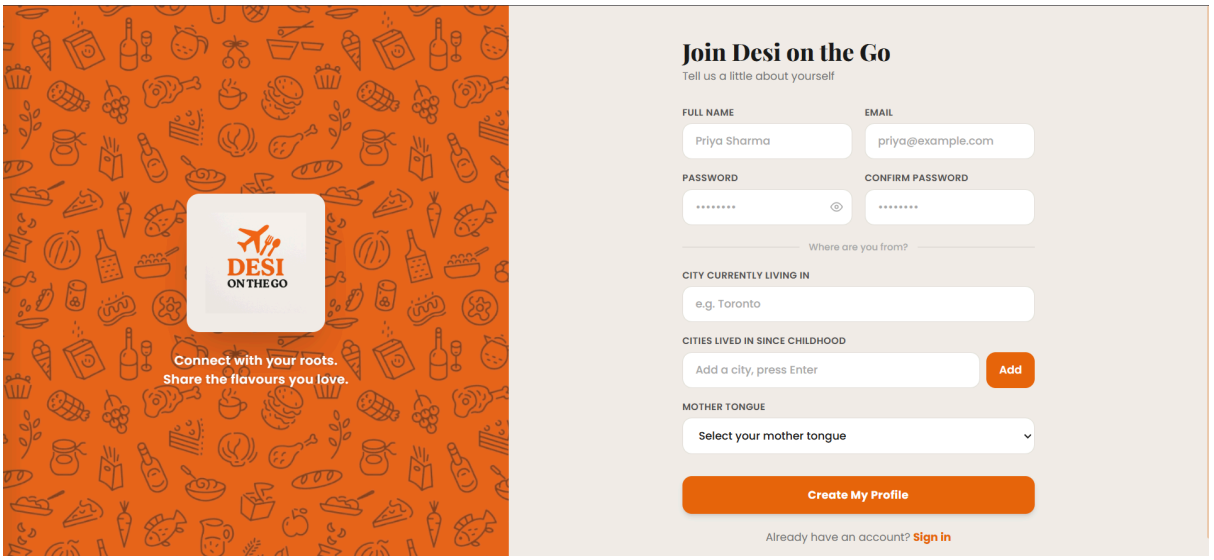
3.2 Module Hierarchy Diagram



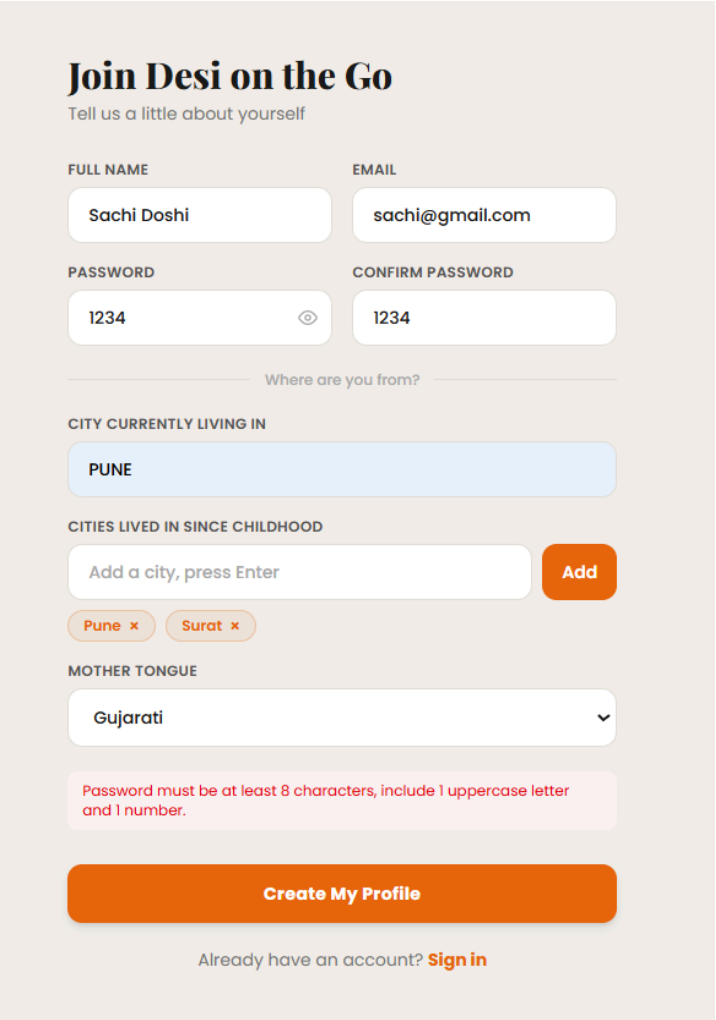
3.3 Sample Input and Output Screens



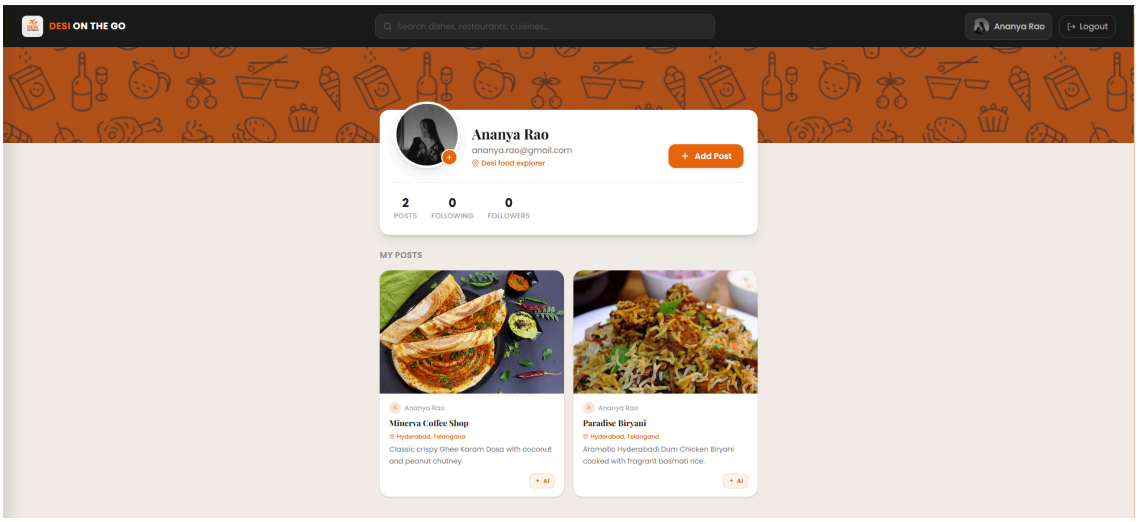
Login Page



Signup page



Signup page (with password validation)



Profile

3	29fea6b8-6548-4a0a-9de0-1824eecd30...	Pranjali	pranjali@example.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	Maharashtra	{G
4	33672c7b-84e3-4f6c-a749-c668426321...	Manjusha Athavale	m@gmail.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	PUNE	{}
5	351b710b-ddb1-4468-8cf4-4ace42dd3a...	harsh ad	harsh@gmail.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	SURAT	{}
6	35666af4-4b25-4406-9517-c159c94430...	Sachi Doshi	sachi@gmail.com	$2b$12$QNNVOUTTrbCTJQZI3YrDjuDHEpkHTnWyfGWiPDII3HuA2SZClnTT...	PUNE	{P
7	3c536aa7-70d3-4516-ab08-c8c580c37a...	string	pranjali@gmail.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	Maharashtra	{G

User added to database

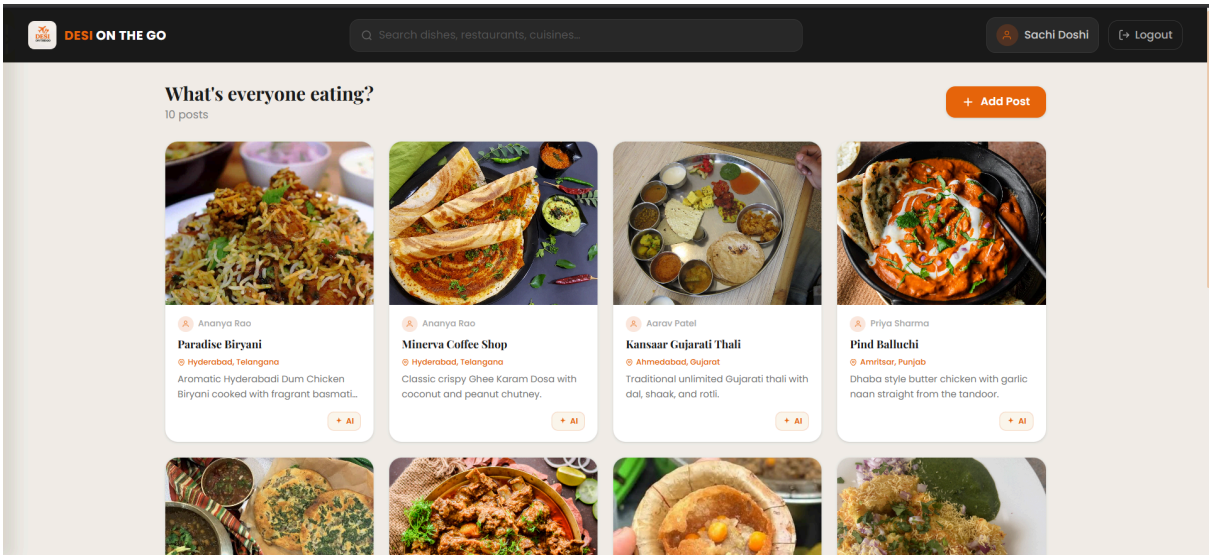
Data Output Messages Notifications

Showing rows: 1 to 13 Page No: 1 of 1

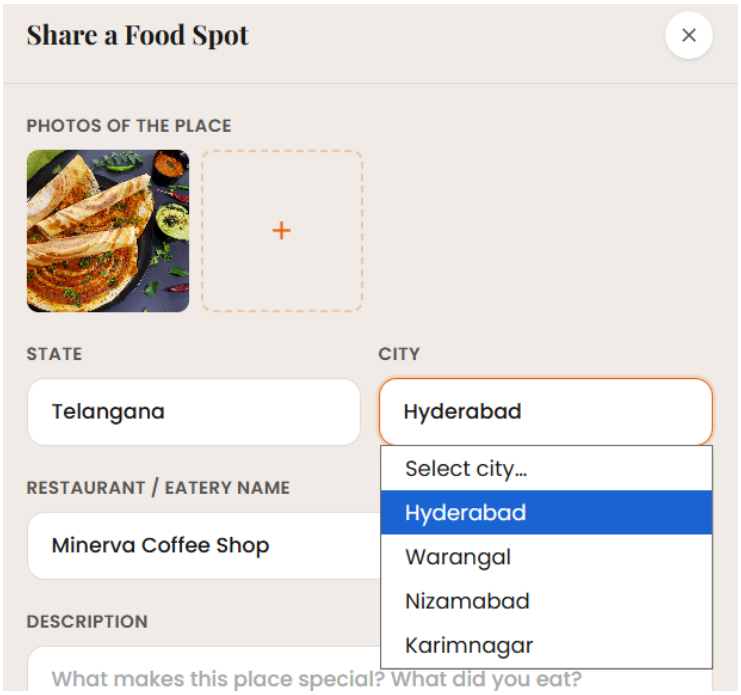
	id [PK] uuid	full_name character varying (100)	email character varying (255)	password_hash character varying (255)	current_city character varying (100)	cities_lived text[]
1	11dc1f2f-9f0f-4ab1-a407-3f9eb45ddbed	Pranav Athavale	pa@gmail.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	PUNE	{}
2	18022f18-f109-441e-b171-070fc3f191b6	Hetvi AD	hetvi@gmail.com	$2b$12$Xy56VRG.crNEKx5wekkln.vGOcOaxPQ8VuupU8UcGQF4AZsy6Olue	Pune	{Surat,Ahemdaba...
3	29fea6b8-6548-4a0a-9de0-1824eecd30...	Pranjali	pranjali@example.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	Maharashtra	{Gujrat}
4	33672c7b-84e3-4f6c-a749-c668426321...	Manjusha Athavale	m@gmail.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	PUNE	{}
5	351b710b-ddb1-4468-8cf4-4ace42dd3a...	harsh ad	harsh@gmail.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	SURAT	{}
6	35666af4-4b25-4406-9517-c159c94430...	Sachi Doshi	sachi@gmail.com	$2b$12$QNNVOUTTrbCTJQZI3YrDjuDHEpkHTnWyfGWiPDII3HuA2SZClnTT...	PUNE	{Pune,Surat}
7	3c536aa7-70d3-4516-ab08-c8c580c37a...	string	pranjali@gmail.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	Maharashtra	{Gujrat}
8	4d3224cb-39b4-4ede-848f-91ae006da1...	Sunil AD	sunil@gmail.com	$2b$12$/GVvv.eLyBXr9RHRi3aMMO.t8FGmSh7MCOGs4YR3r80WC9ZwPy...	Surat	{Pune}
9	a363a401-0269-437a-aa98-06c60a144...	Aarav Patel	aarav.patel@gmail.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	Gujarat	[null]
10	a363a401-0269-437a-aa98-06c60a144...	Pranjali Athavale	p@gmail.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	PUNE	{}
11	b474b502-1370-488b-bb09-17d71b258...	Priya Sharma	priya.sharma@gmail.co...	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	Punjab	[null]
12	c585c603-2481-499c-cc10-28e82c3699...	Rohan Verma	rohan.verma@gmail.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	West Bengal	[null]
13	d696d704-3592-500d-dd21-39f93d4700...	Ananya Rao	ananya.rao@gmail.com	$2b$12$QlXyQxChT/3zBlhyCx/cLO/LmpbvY2/WfkwFyVwqMmKRhc3mRG...	Telangana	[null]

current_city character varying (100)	cities_lived text[]	mother_tongue character varying (50)	profile_photo_url text	created_at timestamp with time zone
PUNE	{}	Gujrati	[null]	2026-10-19 22:49:14.775331+05:...
Pune	{Surat,Ahemdaba...	Gujarati	[null]	2026-10-28 00:11:57.528887+05:...
Maharashtra	{Gujrat}	Marathi	string	2026-10-18 18:03:37.933092+05:...
PUNE	{}	Marathi	[null]	2026-10-21 12:29:37.859362+05:...
SURAT	{}	Gujarati	[null]	2026-10-27 22:33:45.632329+05:...
PUNE	{Pune,Surat}	Gujarati	[null]	2026-10-28 09:38:20.233334+05:...
Maharashtra	{Gujrat}	Marathi	[null]	2026-10-19 19:19:01.332712+05:...
Surat	{Pune}	Gujarati	[null]	2026-10-28 00:18:33.572768+05:...
Gujarat	[null]	Gujarati	[null]	2026-10-27 21:53:51.22472+05:30
PUNE	{}	Marathi	[null]	2026-10-19 21:46:32.590747+05:...
Punjab	[null]	Punjabi	[null]	2026-10-27 21:53:51.22472+05:30
West Bengal	[null]	Bengali	[null]	2026-10-27 21:53:51.22472+05:30
Telangana	[null]	Telugu	[null]	2026-10-27 21:53:51.22472+05:30

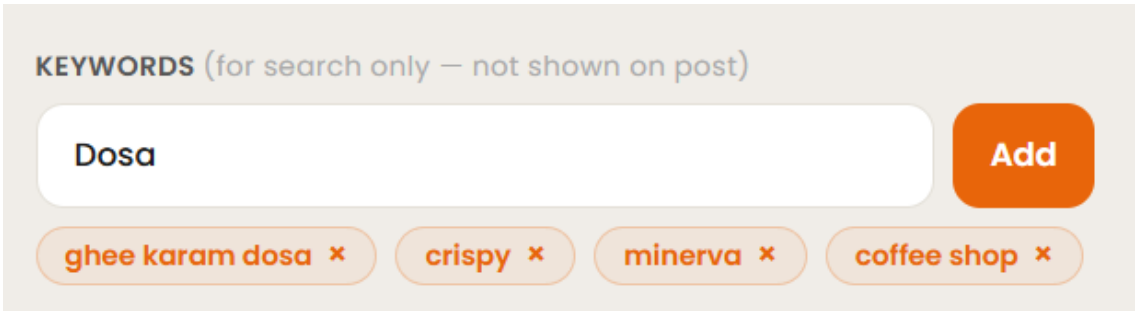
User db



Home page (Midea feed)

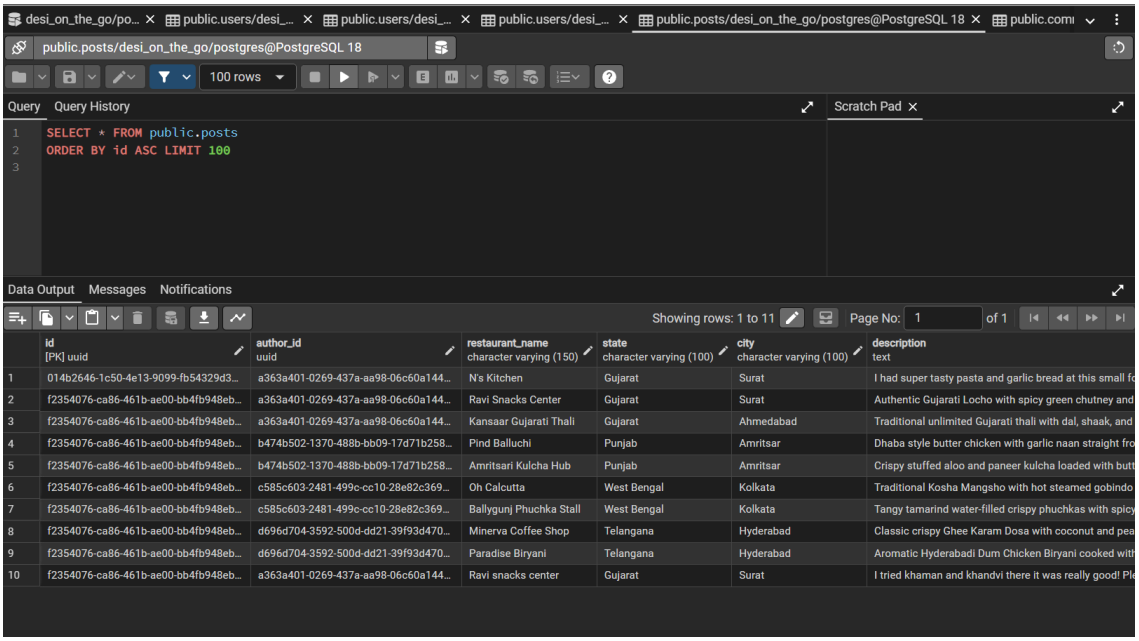


State and city dropdown boxes



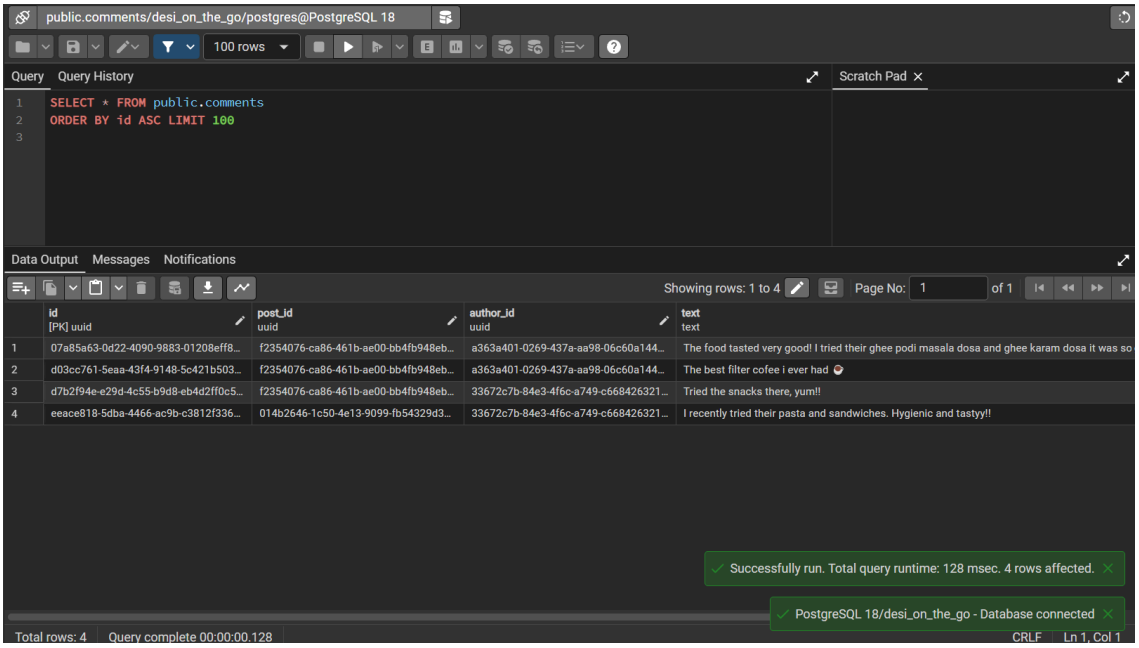


Keywords added to make the search better

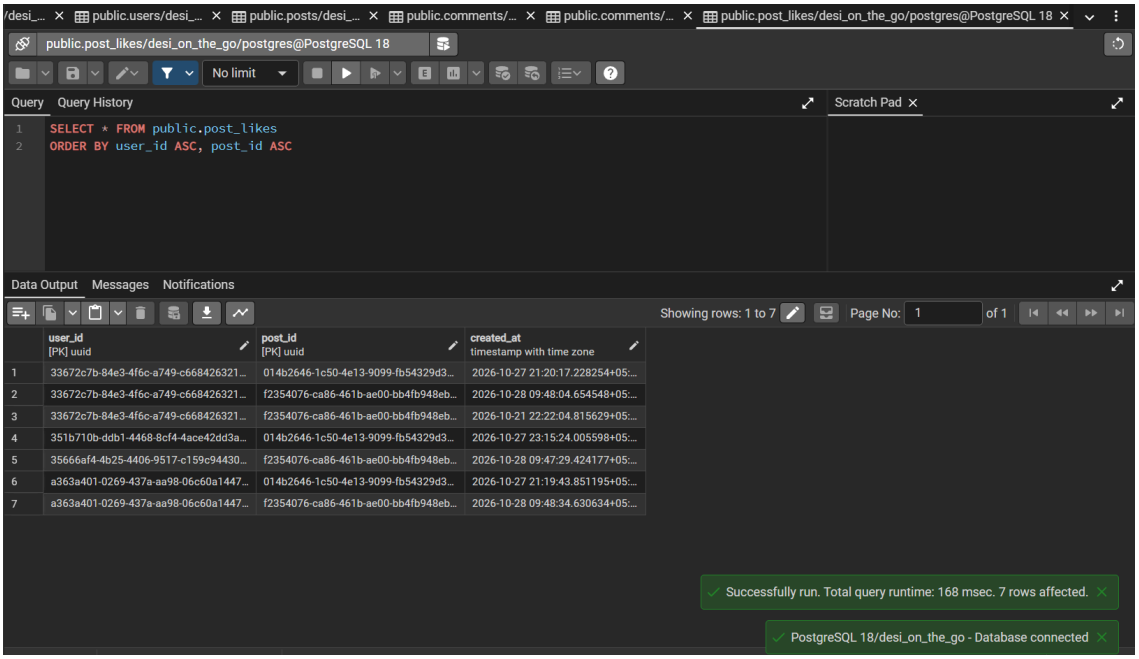


	id [PK] uuid	author_id uuid	restaurant_name character varying (150)	state character varying (100)	city character varying (100)	description text
1	014b2646-1c50-4e13-9099-fb54329d3...	a363a401-0269-437a-aa98-06c60a144...	N's Kitchen	Gujarat	Surat	I had super tasty pasta and garlic bread at this small fo
2	f2354076-ca86-461b-ae00-bb4fb948eb...	a363a401-0269-437a-aa98-06c60a144...	Ravi Snacks Center	Gujarat	Surat	Authentic Gujarati Locho with spicy green chutney and
3	f2354076-ca86-461b-ae00-bb4fb948eb...	a363a401-0269-437a-aa98-06c60a144...	Kansaar Gujarati Thali	Gujarat	Ahmedabad	Traditional unlimited Gujarati thali with dal, shaak, and
4	f2354076-ca86-461b-ae00-bb4fb948eb...	b474b502-1370-488b-bb09-17d71b258...	Pind Balluchi	Punjab	Amritsar	Dhaba style butter chicken with garlic naan straight fro
5	f2354076-ca86-461b-ae00-bb4fb948eb...	b474b502-1370-488b-bb09-17d71b258...	Amritsari Kulcha Hub	Punjab	Amritsar	Crispy stuffed aloo and paneer kulcha loaded with butt
6	f2354076-ca86-461b-ae00-bb4fb948eb...	c585c603-2481-499c-cc10-28e82c369...	Oh Calcutta	West Bengal	Kolkata	Traditional Kosha Mangsho with hot steamed gobindo
7	f2354076-ca86-461b-ae00-bb4fb948eb...	c585c603-2481-499c-cc10-28e82c369...	Ballygunj Phuchka Stall	West Bengal	Kolkata	Tangy tamarind water-filled crispy phuchkas with spicy
8	f2354076-ca86-461b-ae00-bb4fb948eb...	d696d704-3592-500d-dd21-39f93d470...	Minerva Coffee Shop	Telangana	Hyderabad	Classic crispy Ghee Karam Dosa with coconut and pea
9	f2354076-ca86-461b-ae00-bb4fb948eb...	d696d704-3592-500d-dd21-39f93d470...	Paradise Biryani	Telangana	Hyderabad	Aromatic Hyderabadi Dum Chicken Biryani cooked with
10	f2354076-ca86-461b-ae00-bb4fb948eb...	a363a401-0269-437a-aa98-06c60a144...	Ravi snacks center	Gujarat	Surat	I tried khaman and khandvi there it was really good! Pl

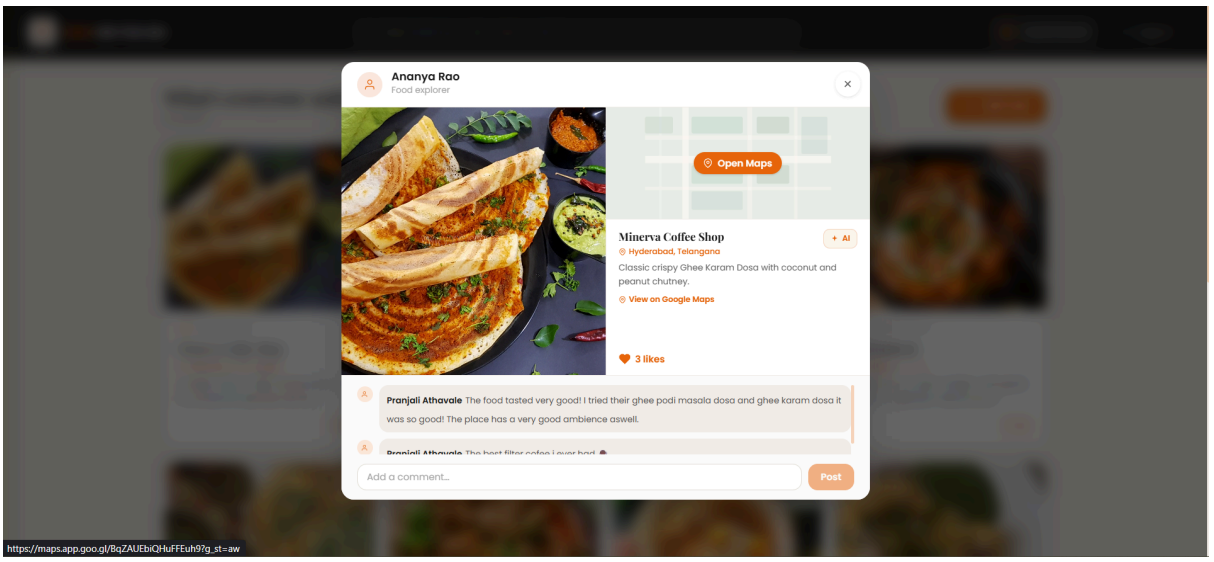
Posts db



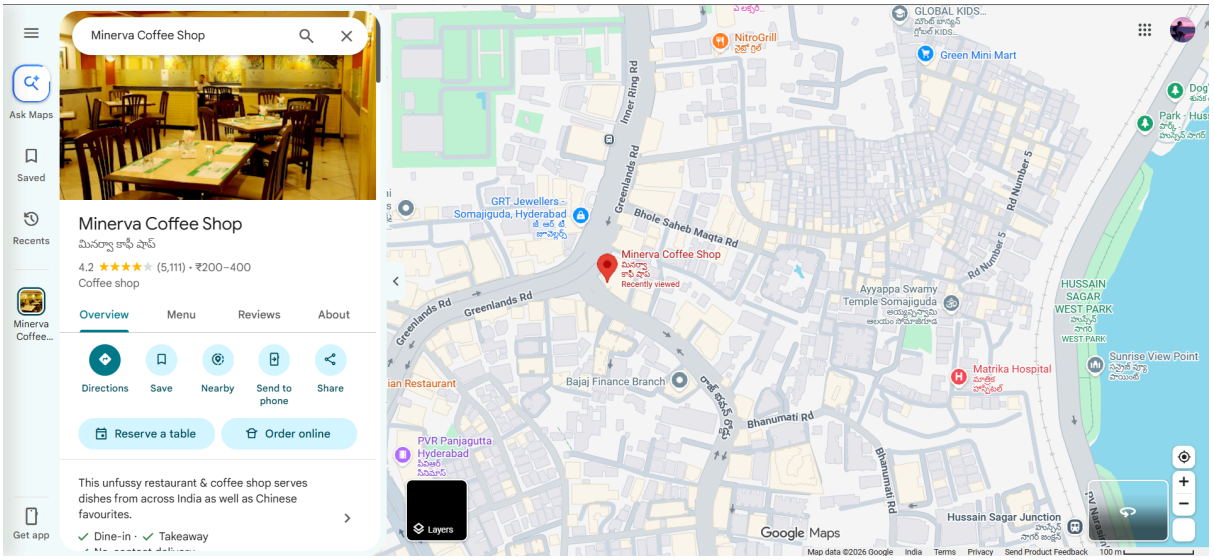
Comments db



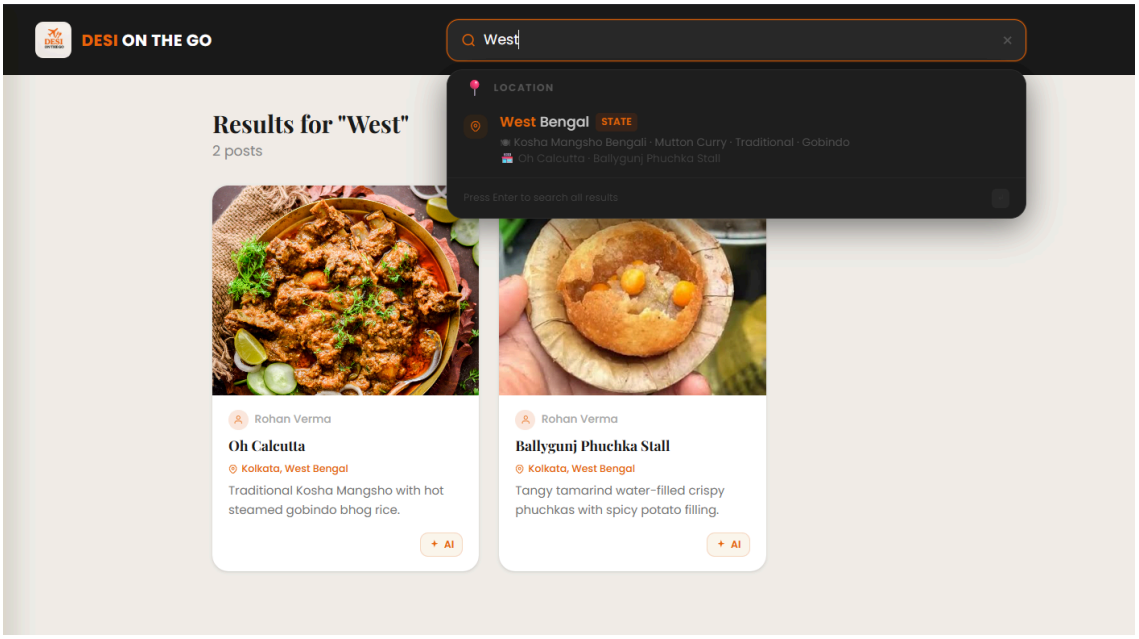
Post_likes db



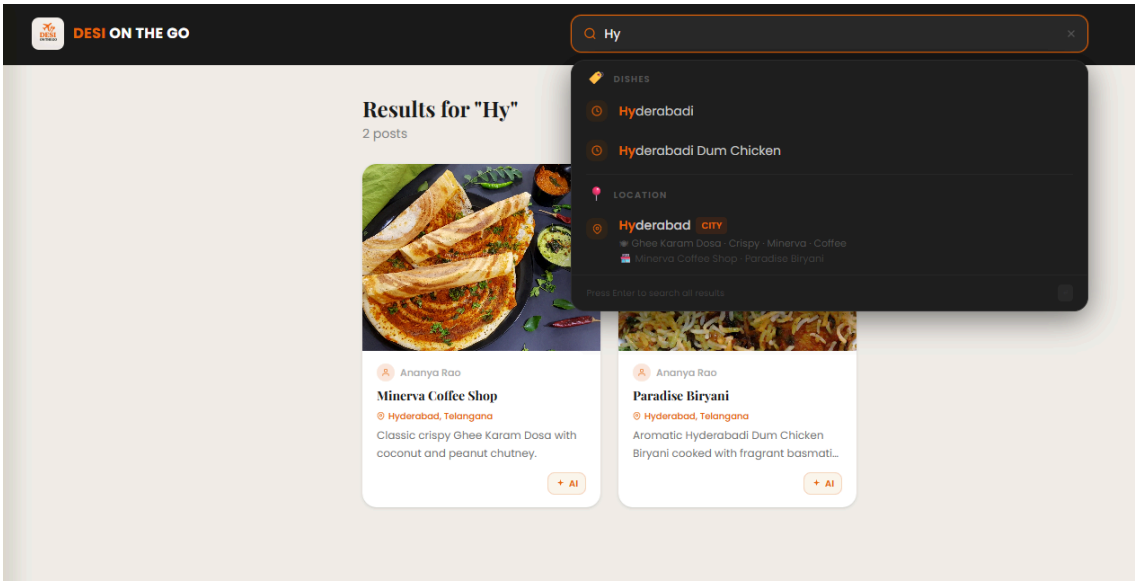
Post (likes comments and google map link)



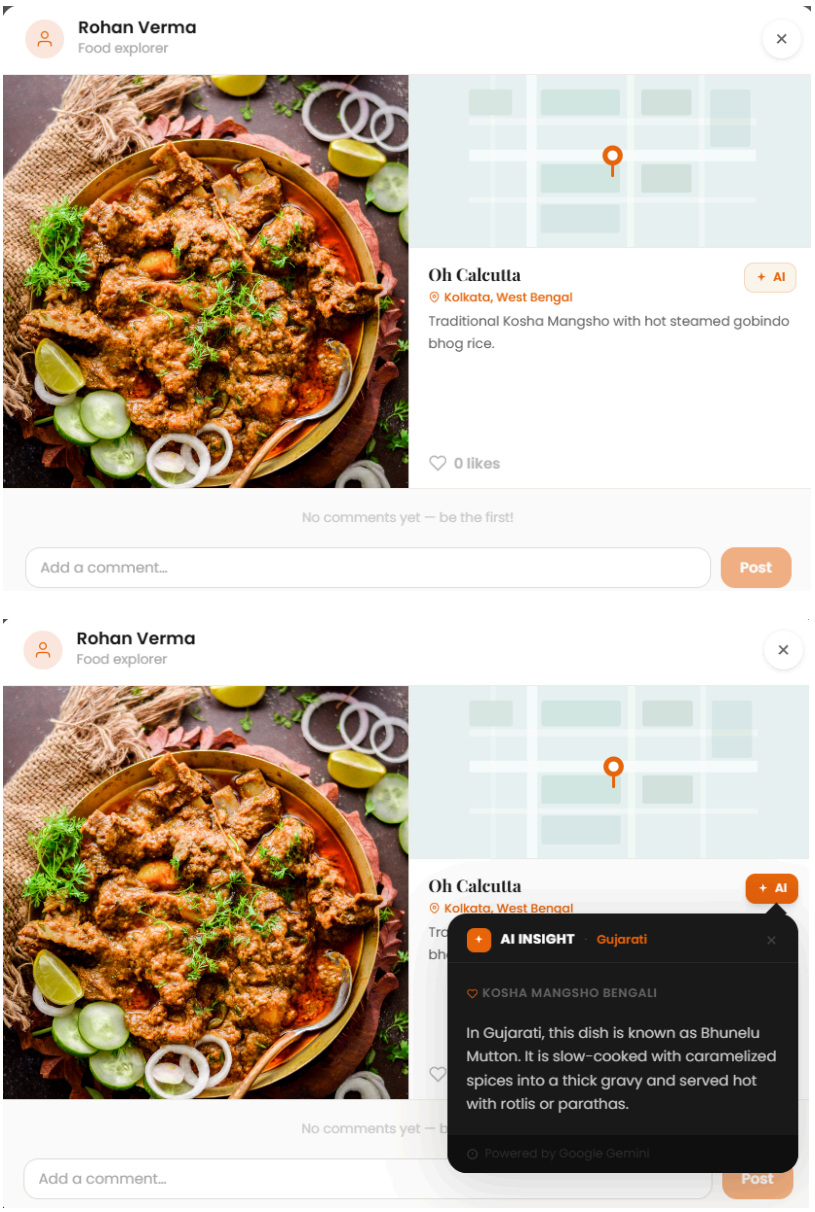
Google map link



Search Based on State



Search based on city or dish



Gemini AI feature

(which converts the post dish into what is it called in native language and gives cultural background significance)

4. Coding

4.1 Code snippets

posts_router.py > ...

```python
from fastapi import APIRouter, Depends, status, Query
from sqlalchemy.orm import Session
from sqlalchemy import or_, cast, Text
from typing import List, Optional
import uuid
from .database import get_db
from . import models, schemas, auth

router = APIRouter(prefix="/api/posts", tags=["Posts & Discovery"])

@router.post("/", response_model=schemas.PostResponse, status_code=status.HTTP_201_CREATED)
def create_post(
    post: schemas.PostCreate,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(auth.get_current_user)
):
    new_post = models.Post(**post.model_dump(), author_id=current_user.id)
    db.add(new_post)
    db.commit()
    db.refresh(new_post)

    # Format comments (will be empty for a new post)
    formatted_comments = [
        schemas.CommentResponse(
            id=c.id,
            author=c.author.full_name if c.author else "Unknown",
            text=c.text
        ) for c in new_post.comments
    ]

    return {
        "id": new_post.id,
        "author_id": new_post.author_id,
        "authorName": current_user.full_name,
        "authorPhoto": current_user.profile_photo_url,
        "motherTongue": current_user.mother_tongue,
        "restaurantName": new_post.restaurant_name,
        "state": new_post.state,
        "city": new_post.city,
        "description": new_post.description,
        "keywords": new_post.keywords,
        "mapLink": new_post.map_link,
        "photos": new_post.photos,
        "likes": 0,
        "liked": False,
        "comments": formatted_comments,
        "created_at": new_post.created_at
    }

@router.get("/", response_model=List[schemas.PostResponse])
def get_posts(
    db: Session = Depends(get_db),
    search: Optional[str] = Query(None, description="Search by restaurant name, city, state, or keywords"),
    current_user: Optional[models.User] = Depends(auth.get_current_user_optional) if hasattr(auth, 'get_current_user_optional') else None
```

```python
@router.post("/{post_id}/like", status_code=status.HTTP_200_OK)
def like_post(
    post_id: uuid.UUID,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(auth.get_current_user)
):
    post = db.query(models.Post).filter(models.Post.id == post_id).first()
    if not post:
        from fastapi import HTTPException
        raise HTTPException(status_code=404, detail="Post not found")

    existing_like = db.query(models.PostLike).filter(
        models.PostLike.post_id == post_id,
        models.PostLike.user_id == current_user.id
    ).first()

    if not existing_like:
        new_like = models.PostLike(post_id=post_id, user_id=current_user.id)
        db.add(new_like)
        db.commit()

    return {"message": "Post liked successfully"}


@router.delete("/{post_id}/like", status_code=status.HTTP_200_OK)
def unlike_post(
    post_id: uuid.UUID,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(auth.get_current_user)
):
    existing_like = db.query(models.PostLike).filter(
        models.PostLike.post_id == post_id,
        models.PostLike.user_id == current_user.id
    ).first()

    if existing_like:
        db.delete(existing_like)
        db.commit()

    return {"message": "Post unliked successfully"}


@router.post("/{post_id}/comments", status_code=status.HTTP_201_CREATED)
def create_comment(
    post_id: uuid.UUID,
    comment: schemas.CommentCreate,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(auth.get_current_user)
):
    post = db.query(models.Post).filter(models.Post.id == post_id).first()
    if not post:
```

tsconfig.json | App.tsx | api.ts | vite.config.ts | site.json | index.cjs | temp_clipboard.txt

src > App.tsx > PhotoCollage

```tsx
export default function App() {
  // Check if BOTH the token and saved profile exist
  const savedToken = localStorage.getItem("access_token")
  const savedProfileJson = localStorage.getItem("user_profile")

  const hasValidSession = Boolean(savedToken && savedProfileJson)

  const [view, setView] = useState<View>(hasValidSession ? "home" : "login")

  const [profile, setProfile] = useState<UserProfile>(() => {
    if (savedProfileJson) {
      try {
        return JSON.parse(savedProfileJson)
      } catch {
        // Fallback if JSON is corrupted
      }
    }
    return {
      name: "",
      email: "",
      photo: null,
      motherTongue: "Hindi",
    }
  })

  const [profilePhoto, setProfilePhoto] = useState<string | null>(profile.photo)
  const [posts, setPosts] = useState<Post[]>([])
  const [showGuide, setShowGuide] = useState(true)

```

```tsx
if (view === "login")
  return (
    <LoginPage
      onSignIn={async (email, password) => {
        const user = await login(email, password)
        const newProfile = {
          name: user.name,
          email: user.email,
          photo: user.photo,
          motherTongue: user.motherTongue,
        }
        setProfile(newProfile)
        setProfilePhoto(user.photo)

        // Save both profile and ensure token is set
        localStorage.setItem("user_profile", JSON.stringify(newProfile))

        setView("home")
        void refreshPosts()
      }}
      onGoSignup={() => setView("signup")}
    />
  )
if (view === "signup")
  return (
    <SignupPage
      onComplete={(p) => {
```

5. Limitations of Proposed System

While the current implementation of **Desi on the Go** successfully establishes a robust full-stack foundation for regional culinary discovery, evaluating the prototype in its present state highlights several operational and technical limitations that provide context for future development:

- **Dependence on Crowdsourced & Unverified Content:**
  The current system relies primarily on user-generated posts and public data inputs. In the absence of an administrative verification mechanism, the platform remains vulnerable to unverified venue listings, outdated operational hours, or inaccurate menu offerings submitted by community members.
- **Absence of Direct Merchant Engagement:**
  The prototype operates as a consumer-facing discovery tool with no dedicated interface for business owners. Regional food vendors and traditional street stalls currently lack the ability to claim their listings, update their daily menus in real time, or directly communicate temporary closures and seasonal specials to users.
- **Coarse Categorization & Search Granularity:**
  Content discovery in the current version relies on general regional tags and text-based descriptions. The platform lacks micro-granular dish-level tagging within media uploads, which limits search precision when users attempt to query specific, niche regional recipes (such as *Kosha Mangsho* or *Ghee Karam Dosa*) rather than broad restaurant categories.
- **Static Location Services:**
  While Google Maps is integrated to display static restaurant coordinates, the current architecture does not support dynamic spatial features. Users cannot receive real-time, proximity-based notifications or geofenced alerts when moving near temporary pop-up stalls, cultural food markets, or diaspora culinary events.

6. Proposed Enhancements

Although the current implementation of Desi on the Go serves as a fully functional, high-performance prototype, its underlying architecture is intentionally engineered for modularity and scalability. By leveraging asynchronous backend routing via FastAPI, a structured PostgreSQL schema, and a component-driven React frontend, the application is uniquely positioned to scale seamlessly from a localized prototype into a comprehensive, high-throughput commercial platform. To build upon this foundation and expand its operational scope, several key enhancements are slated for integration in future production releases:

- **Vendor-Side Portal & Management Dashboard:**
  Introducing a dedicated dashboard tailored for restaurant owners and street food vendors to claim profiles, upload live digital menus, update operational hours in real time, and broadcast seasonal specials directly to nearby users.

- **Verified Vendor Tagging & Trust Badges:**
  Implementing a verification badge system to authenticate official stalls and established eateries. Verified businesses will display trust badges, helping users distinguish certified regional kitchens from crowdsourced listings to boost platform credibility.

- **Micro-Granular Dish Tagging & Search Precision:**
  Upgrading media uploads to allow direct tagging of specific signature dishes (such as Kosha Mangsho or Ghee Karam Dosa) within photo collages. This will feed into an advanced search index, enabling users to query exact dishes rather than generic categories.

- **Real-Time Location & Event Alerts:**
  Expanding Google Maps integration to support dynamic geofencing, alerting users when they are near verified pop-up stalls, cultural food festivals, or highly rated diaspora culinary events.

7. Conclusion

The developed platform successfully bridges the gap between food lovers and regional culinary heritage, offering an intuitive, high-performance interface designed specifically for sharing and discovering authentic dining experiences. By integrating cultural context, mother-tongue filtering, dynamic location mapping, and real-time social interactions into a cohesive architecture, the system provides a specialized, community-driven alternative to generic commercial review platforms. The current prototype successfully validates the feasibility and technical efficacy of combining modern asynchronous backend services with type-safe, responsive frontend rendering to cater directly to the unique cultural discovery needs of diaspora communities.

Throughout its implementation, the platform has demonstrated how technical precision achieved through strict RESTful API contracts, Pydantic data validations, and secure JWT session management can directly elevate user experience. Beyond serving as a simple directory of dining spots, the system fosters meaningful engagement by enabling users to document regional culinary traditions, interact with peer reviews through threaded comments and likes, and unlock rich cultural narratives behind traditional dishes using AI-driven insight triggers.

Looking forward, the application's modular and decoupled architectural design provides a highly adaptable foundation for future growth. By incorporating proposed enhancements such as vendor-side profile management, verified business tagging, micro-granular dish categorization, and dynamic geofenced location alerts, the system is exceptionally well-positioned to scale from a localized prototype into a comprehensive, high-throughput commercial ecosystem for global culinary exploration and heritage preservation.

8. Bibliography

- Connolly, T. M., & Begg, C. E. *Database Systems: A Practical Approach to Design, Implementation, and Management*. Pearson Education, 2015.
- FastAPI Documentation. "FastAPI: High performance, easy to learn, fast to code, ready for production." Retrieved from https://fastapi.tiangolo.com, 2026.
- Google AI. "Gemini API Documentation." Retrieved from https://ai.google.dev/docs, 2026.
- Google. *Gemini* (Large language model). Retrieved from https://gemini.google.com, 2026.
- Haverbeke, E. *Eloquent JavaScript: A Modern Introduction to Programming*. No Starch Press, 2018.
- Node.js Documentation. "Node.js v24 API Documentation." Retrieved from https://nodejs.org/docs, 2026.
- Richardson, L., & Ruby, S. *RESTful Web Services*. O'Reilly Media, 2007.
- Supabase Documentation. "PostgreSQL Database and Real-time APIs." Retrieved from https://supabase.com/docs, 2026.
- Uvicorn Documentation. "An ASGI server implementation for Python." Retrieved from https://www.uvicorn.org, 2026.